

DSC 40B

Lecture 19 : BFS,
Part 2

Review: Aggregate Analysis

Question

What is the time complexity of the following code in terms of $|V|$ and $|E|$?

```
for u in graph.nodes:  
    for v in graph.neighbors(u):  
        print("Edge:", u, v)
```

Question

What is the time complexity of the following code in terms of $|V|$ and $|E|$?

```
for u in graph.nodes:  
    for v in graph.neighbors(u):  
        print("Edge:", u, v)
```

How many times do we execute this line?

Question

What is the time complexity of the following code in terms of $|V|$ and $|E|$?

```
for u in graph.nodes:  
    for v in graph.neighbors(u):  
        print("Edge:", u, v)
```

How many times do we execute this line? **E or 2*E**

Question

What is the time complexity of the following code in terms of $|V|$ and $|E|$? $\Theta(V + E)$ #V to cover the case when a graph does not have edges.

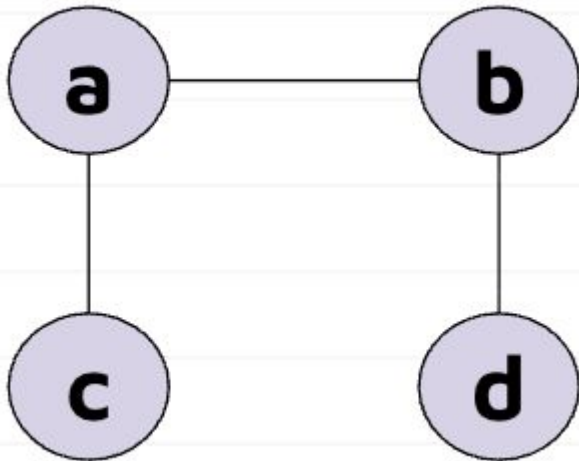
```
for u in graph.nodes:  
    for v in graph.neighbors(u):  
        print("Edge:", u, v)
```

Answer

```
for u in graph.nodes:  
    for v in graph.neighbors(u):  
        print("Edge:", u, v)
```

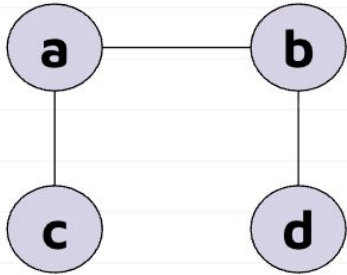
- Time complexity: $\Theta(V + E)$
- The **print** executes:
 - **once** for each edge, if the graph is **directed**.
 - **twice** for each edge, if the graph is **undirected**.

dict-of-sets representation



```
adj = {  
    "a": {"b", "c"},  
    "b": {"a", "d"},  
    "c": {"a"},  
    "d": {"b"}  
}
```

Another Look...



Consider the graph's dict-of-sets representation.

```
adj = {  
    "a": {"b", "c"},  
    "b": {"a", "d"},  
    "c": {"a"},  
    "d": {"b"}  
}
```

```
for u in graph.nodes:  
    for v in graph.neighbors(u):  
        print("Edge:", u, v)
```

```
def full_bfs(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            bfs(graph, node, status)

def bfs(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}
    status[source] = 'pending'
    pending = deque([source])
    # while there are still pending nodes
    while pending:
        u = pending.popleft()
        for v in graph.neighbors(u):
            # explore edge (u,v)
            if status[v] == 'undiscovered':
                status[v] = 'pending'
                # append to right
                pending.append(v)
        status[u] = 'visited'
```

```
def full_bfs(graph):  
    status = {node: 'undiscovered' for node in graph.nodes}  
    for node in graph.nodes:  
        if status[node] == 'undiscovered':  
            bfs(graph, node, status)
```

$\Theta(?)$

```
def bfs(graph, source):  
    """Start a BFS at `source`."""  
    status = {node: 'undiscovered' for node in graph.nodes}  
    status[source] = 'pending'  
    pending = deque([source])  
    # while there are still pending nodes  
    while pending:  
        u = pending.popleft()  
        for v in graph.neighbors(u):  
            # explore edge (u,v)  
            if status[v] == 'undiscovered':  
                status[v] = 'pending'  
                # append to right  
                pending.append(v)  
        status[u] = 'visited'
```

```
def full_bfs(graph):  
    status = {node: 'undiscovered' for node in graph.nodes}  
    for node in graph.nodes:  
        if status[node] == 'undiscovered':  
            bfs(graph, node, status)
```

$\Theta(V)$

```
def bfs(graph, source):  
    """Start a BFS at `source`."""  
    status = {node: 'undiscovered' for node in graph.nodes}  
    status[source] = 'pending'  
    pending = deque([source])  
    # while there are still pending nodes  
    while pending:  
        u = pending.popleft()  
        for v in graph.neighbors(u):  
            # explore edge (u,v)  
            if status[v] == 'undiscovered':  
                status[v] = 'pending'  
                # append to right  
                pending.append(v)  
        status[u] = 'visited'
```

```
def full_bfs(graph):  
    status = {node: 'undiscovered' for node in graph.nodes}  
    for node in graph.nodes:  
        if status[node] == 'undiscovered':  
            bfs(graph, node, status)
```

$\Theta(V)$

```
def bfs(graph, source):  
    """Start a BFS at `source`."""  
    status = {node: 'undiscovered' for node in graph.nodes}
```

```
    status[source] = 'pending'  
    pending = deque([source])
```

How many times do we call this line?

```
    # while there are still pending nodes  
    while pending:  
        u = pending.popleft()  
        for v in graph.neighbors(u):  
            # explore edge (u,v)  
            if status[v] == 'undiscovered':  
                status[v] = 'pending'  
            # append to right  
            pending.append(v)  
        status[u] = 'visited'
```

```
def full_bfs(graph):  
    status = {node: 'undiscovered' for node in graph.nodes}  
    for node in graph.nodes:  
        if status[node] == 'undiscovered':  
            bfs(graph, node, status)
```

$\Theta(V)$

```
def bfs(graph, source):  
    """Start a BFS at `source`."""  
    status = {node: 'undiscovered' for node in graph.nodes}
```

```
    status[source] = 'pending'  
    pending = deque([source])
```

```
    # while there are still pending nodes  
    while pending:
```

```
        u = pending.popleft()
```

```
        for v in graph.neighbors(u):
```

```
            # explore edge (u,v)
```

```
            if status[v] == 'undiscovered':
```

```
                status[v] = 'pending'
```

```
                # append to right
```

```
                pending.append(v)
```

```
    status[u] = 'visited'
```

What is the graph if my goal is to make full_bfs to call bfs as many times as possible?

```
def full_bfs(graph):  
    status = {node: 'undiscovered' for node in graph.nodes}  
    for node in graph.nodes:  
        if status[node] == 'undiscovered':  
            bfs(graph, node, status)
```

$\Theta(V)$

```
def bfs(graph, source):  
    """Start a BFS at `source`."""  
    status = {node: 'undiscovered' for node in graph.nodes}
```

```
    status[source] = 'pending'  
    pending = deque([source])
```

No more than V, can be ignored

```
    # while there are still pending nodes
```

```
    while pending:
```

```
        u = pending.popleft()
```

```
        for v in graph.neighbors(u):
```

```
            # explore edge (u,v)
```

```
            if status[v] == 'undiscovered':
```

```
                status[v] = 'pending'
```

```
                # append to right
```

```
                pending.append(v)
```

```
    status[u] = 'visited'
```

```
def full_bfs(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            bfs(graph, node, status)
```

$\Theta(V)$

```
def bfs(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}
```

```
    status[source] = 'pending'
    pending = deque([source])
```

No more than V, can be ignored

```
    # while there are still pending nodes
```

```
    while pending:
        u = pending.popleft()
        for v in graph.neighbors(u):
            # explore edge (u,v)
            if status[v] == 'undiscovered':
                status[v] = 'pending'
                # append to right
                pending.append(v)
        status[u] = 'visited'
```

What line of code is executed the most?

```
def full_bfs(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            bfs(graph, node, status)
```

$\Theta(V)$

```
def bfs(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}
```

```
    status[source] = 'pending'
    pending = deque([source])
```

No more than V, can be ignored

```
    # while there are still pending nodes
```

```
    while pending:
```

```
        u = pending.popleft()
```

```
        for v in graph.neighbors(u):
```

```
            # explore edge (u,v)
```

```
            if status[v] == 'undiscovered':
```

```
                status[v] = 'pending'
```

```
                # append to right
```

```
                pending.append(v)
```

```
    status[u] = 'visited'
```

```
def full_bfs(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            bfs(graph, node, status)
```

$\Theta(V)$

```
def bfs(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}
```

```
    status[source] = 'pending'
    pending = deque([source])
```

No more than V, can be ignored

```
    # while there are still pending nodes
    while pending:
```

```
        u = pending.popleft()
        for v in graph.neighbors(u):
            # explore edge (u,v)
```

print(u,v) how many times?

```
            if status[v] == 'undiscovered':
```

```
                status[v] = 'pending'
```

```
                # append to right
```

```
                pending.append(v)
```

```
    status[u] = 'visited'
```

```
def full_bfs(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            bfs(graph, node, status)
```

$\Theta(V)$

```
def bfs(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}
```

```
    status[source] = 'pending'
    pending = deque([source])
```

No more than V, can be ignored

```
    # while there are still pending nodes
    while pending:
```

```
        u = pending.popleft()
```

```
        for v in graph.neighbors(u):
```

```
            # explore edge (u,v)
```

print(u,v) |E| or 2|E| in aggregate

```
            if status[v] == 'undiscovered':
```

```
                status[v] = 'pending'
```

```
                # append to right
```

```
                pending.append(v)
```

```
    status[u] = 'visited'
```

Question

Suppose we run a full_bfs is called on a graph with no edges, does the comparison line gets executed?

A: Yes, many times

B: Yes, but once

C: No

D: I do not know :(

Question

Suppose bfs is called on an undirected graph using source node u .

If u is part of a connected component with nodes V_1 and edges E_1 , what is the time complexity of the call to bfs?

A: $\Theta(E_1)$

B: $\Theta(V_1)$

C: $\Theta(E_1 + V_1)$

D: $\Theta(V_1 * E_1)$

E: Something else

Answer

- Time complexity: $\Theta(V_1 + E_1)$
- bfs explores all nodes and edges in the connected component.

Time Complexity of Full BFS

- `full_bfs` calls `bfs` **once** for **each** connected component.
- **Time complexity:**

$$\begin{aligned} & \Theta((V_1 + E_1) + (V_2 + E_2) + \dots + (V_k + E_k)) \\ &= \Theta((V_1 + V_2 + \dots + V_k) + (E_1 + E_2 + \dots + E_k)) \\ &= \Theta(V + E) \end{aligned}$$

Time Complexity

- Full BFS takes $\Theta(V + E)$
- Why not just $\Theta(E)$?
- $\Theta(V + E)$ works for *all graphs*.
 - If we know more about the number of edges, we might be able to simplify.
 - E.g., if the graph is **complete**, $E = \Theta(V^2)$, so time complexity is $\Theta(V + V^2) = \Theta(V^2)$.



Thank you!

Do you have any questions?

CampusWire!